Learning Computational Research Through a Spring-Network Model of Cell-Cell Adhesion: An RET Project

The Research Experiences for Teachers (RET) program provides educators with opportunities to engage in authentic research while developing classroom resources that bring current scientific practices into K–12 education. As science increasingly integrates computational methods with traditional laboratory investigations, it is important for students to understand not only how experiments are performed at the bench, but also how computational models are used to investigate scientific questions.

This project consists of three complementary components designed to introduce computational research and demonstrate how it can be incorporated into science education. The first component is a lesson comparing computational science with traditional bench science, emphasizing the strengths, limitations, and applications of each approach. The second component is a lesson describing the foundational knowledge and tools needed to begin a computational research project, including problem formulation, model development, programming, and data analysis. The third component is a simple computational research project that applies these concepts by modeling cell-cell adhesions in epithelial tissue using a two-dimensional spring-network model.

Together, these components illustrate the progression from understanding the role of computational science, to learning the skills required to perform computational research, to applying those skills in an authentic research investigation. While the computational model presented here is intentionally simple, it demonstrates the complete research process of developing a scientific question, constructing a computational model, analyzing simulation results, and interpreting those results in a biological context. The overall goal is to provide both educators and students with an accessible introduction to computational research and its growing role in modern science.

Part 1- The Research

Part 2- Lesson 1

Part 3- Lesson 2

Presentation Slides

Link to Loom Presentation

Part 1– The Research

Introduction

Tissues depend on connections between neighboring cells to maintain their shape and resist external forces. These cell-cell adhesions help transmit forces across the tissue. When the adhesions become weaker or are lost, the tissue may become less stiff and more easily deformed.

Studying tissue mechanics experimentally can be difficult because living tissues contain many interacting cells and materials. Computational models provide a simpler way to study how individual factors, such as adhesion strength and adhesion loss, affect tissue behavior.

In this study, a two-dimensional spring network was used as a simplified model of epithelial tissue. Cells were represented as nodes, and cell-cell adhesions were represented as linear springs that obey Hooke's Law. The model was used to test how weaker springs and the random removal of springs affected force transmission, effective tissue stiffness, functional integrity, and mathematical stability. It was hypothesized that weakening or removing springs would reduce tissue stiffness and that extensive spring loss would cause the network to lose its ability to transmit forces.

Methods

A two-dimensional spring-network model was developed in MATLAB to represent a simplified epithelial tissue. Six cells were represented as nodes arranged in two columns and three rows. Cell-cell adhesions were represented by 11 axial springs, consisting of three horizontal, four vertical, and four diagonal connections. Each spring obeyed Hooke's Law, meaning that the force generated by a spring was directly proportional to its change in length.

The left-side nodes were fixed in both the horizontal and vertical directions to simulate an anchored tissue. A horizontal displacement was applied to the three nodes on the right side while allowing their vertical movement. Eleven displacement values, ranging from 0 to 0.10 model units, were tested. For each displacement, a global stiffness matrix was assembled, and the unknown node displacements were calculated by solving the equations for mechanical equilibrium. The total reaction force at the displaced boundary was then calculated, and the effective tissue stiffness was determined as the slope of the resulting force-displacement relationship.

Two independent variables were investigated. The first was **cell-cell adhesion strength**, represented by spring stiffness. A healthy tissue model used a spring stiffness of 100, while a weak-adhesion model used a spring stiffness of 40. The second independent variable was **cell-cell adhesion loss**, simulated by randomly removing springs from the healthy network. Spring removal ranged from 0% to 90% in 10% increments. At each level of spring removal, 100 random networks were generated while all remaining springs retained the healthy stiffness value. A fixed random-number seed (`rng(1)`) was used to ensure that the simulations were reproducible.

The dependent variables were reaction force, effective tissue stiffness, the fraction of mathematically stable networks, and the fraction of functionally intact networks. A network was classified as

mathematically stable when its stiffness matrix remained solvable after spring removal. A network was considered functionally intact if it was mathematically stable and retained at least 10% of the effective stiffness of the healthy network. For each spring-removal level, the mean effective stiffness, standard deviation, fraction of mathematically stable networks, and fraction of functionally intact networks were calculated.

Results

Figure 1 Force–displacement response of the spring network for healthy tissue and tissue with weakened cell-cell adhesions. Both conditions exhibit a linear relationship between applied displacement and reaction force, consistent with Hooke's Law. However, the weak adhesion model produces substantially lower reaction forces at all displacement levels, indicating a reduced effective tissue stiffness and diminished resistance to tensile deformation.

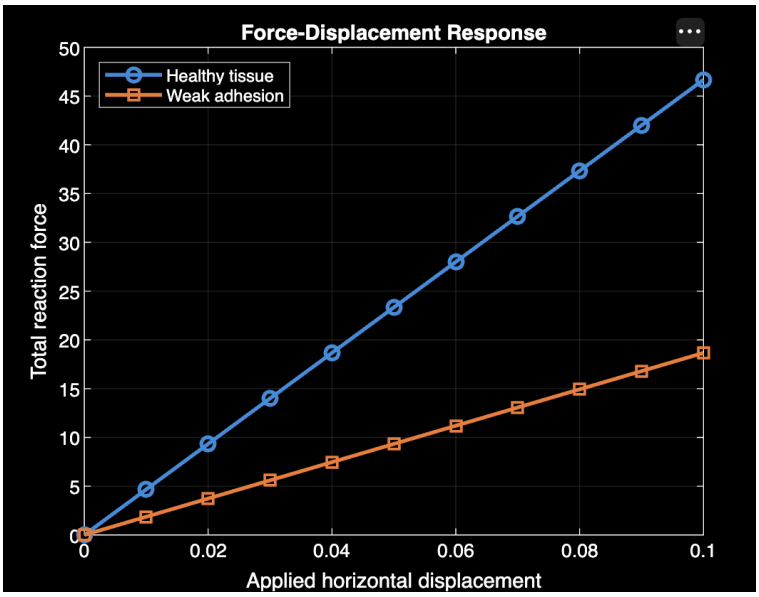


Figure 2 Comparison of the effective stiffness of healthy tissue and tissue with weakened cell-cell adhesions. Reducing the spring stiffness, which represents weaker cell-cell adhesion, results in a substantial decrease in the effective stiffness of the tissue. This demonstrates that adhesion strength is a major determinant of the tissue's ability to resist tensile deformation.

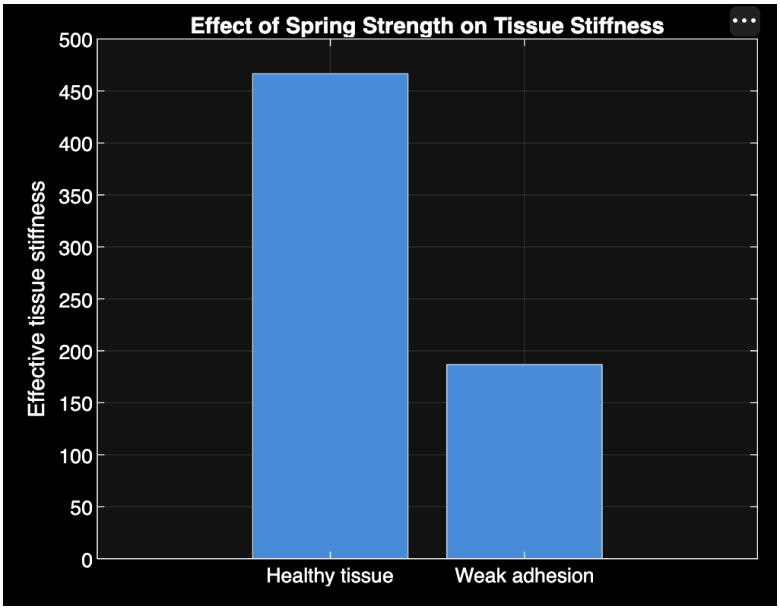


Figure 3 Effect of progressive spring (cell-cell adhesion) loss on the effective stiffness of the tissue. As the percentage of removed springs increases, the mean effective tissue stiffness decreases, indicating reduced resistance to tensile deformation. Error bars represent the variability among random spring-removal simulations, demonstrating that the specific pattern of adhesion loss influences the mechanical response, particularly at intermediate levels of damage.

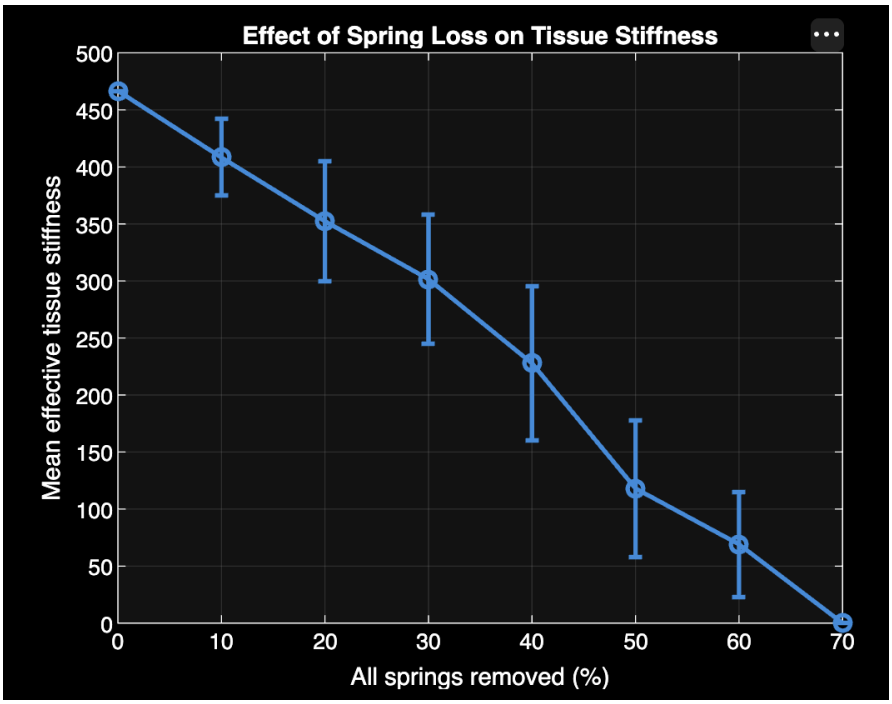
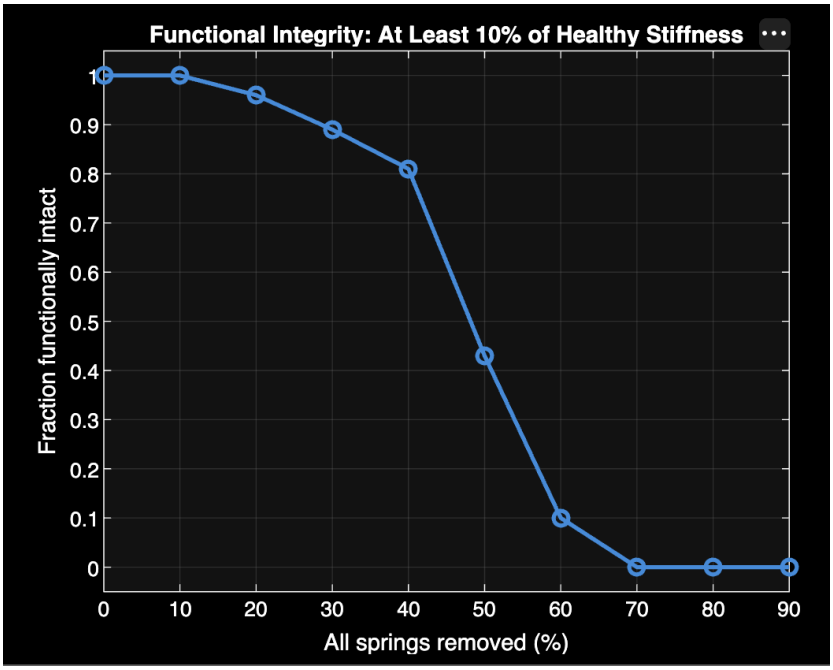


Figure 4 Functional integrity of the spring network as a function of spring (cell-cell adhesion) loss. A network was considered functionally intact if its effective stiffness remained at least 10% of that of the healthy tissue. Functional integrity remains high with limited adhesion loss but declines sharply after approximately 40–50% of the springs are removed, indicating a critical threshold beyond which the tissue can no longer maintain its mechanical function.



Discussion

The spring-network model simulated the mechanical behavior of a simplified epithelial tissue under tensile loading. As expected for a network of linear springs, the relationship between applied displacement and reaction force was linear for both the healthy and weak-adhesion models (Figure 1). This result is consistent with Hooke's Law and indicates that the model behaved as intended.

Reducing the spring stiffness, which represented weaker cell-cell adhesions, produced a substantial decrease in effective tissue stiffness (Figure 2). This finding demonstrates that the strength of cell-cell adhesions plays an important role in determining how well a tissue resists deformation. Although the overall geometry of the tissue remained unchanged, weaker adhesions reduced the ability of the network to transmit forces across neighboring cells.

The adhesion-loss simulations further demonstrated that tissue stiffness depends not only on the strength of individual adhesions but also on the number of connections between cells. As progressively more springs were removed, the average effective stiffness of the network steadily decreased (Figure 3). The increasing error bars at intermediate levels of spring removal indicate that the specific pattern of adhesion loss also influenced tissue mechanics. Networks with the same percentage of removed springs could behave differently depending on which particular connections remained.

Although tissue stiffness declined gradually, functional integrity remained relatively high until approximately 40–50% of the springs had been removed (Figure 4). Beyond this point, the ability of the network to resist stretching decreased rapidly, suggesting the presence of a critical connectivity threshold. This behavior indicates that tissues may tolerate modest amounts of adhesion loss before experiencing a rapid decline in mechanical function.

This model has several limitations. Cells were represented as point nodes connected by identical linear springs, and the tissue was modeled in two dimensions. Real epithelial tissues contain cells with complex shapes, nonlinear mechanical properties, and adhesions that vary in strength and

organization. In addition, living tissues actively remodel their cell-cell junctions and generate contractile forces, processes that were not included in this model. Therefore, the simulations should be viewed as illustrating general mechanical principles rather than predicting the behavior of a specific biological tissue.

Despite these simplifications, the model demonstrates how computational simulations can be used to investigate the mechanical consequences of changes in cell-cell adhesion. The results suggest that both adhesion strength and network connectivity are important determinants of tissue stiffness and mechanical integrity. This approach could be extended in future studies to include larger tissues, nonlinear material behavior, heterogeneous adhesion strengths, or experimental data for validation.

Conclusion

A two-dimensional spring network model was developed to simulate how epithelial tissue responds to stretching. The model showed a linear relationship between applied displacement and reaction force, consistent with Hooke's Law. When the spring stiffness was reduced to represent weaker cell-cell adhesions, the tissue became less stiff and resisted stretching less effectively.

As springs were progressively removed, the effective tissue stiffness also decreased. The model showed that the tissue could tolerate small amounts of adhesion loss, but its mechanical function declined rapidly after about 40–50% of the springs were removed. With extensive spring loss, the network eventually became mechanically unstable because there were no longer enough connections to transmit forces across the tissue.

Overall, the results show that both the strength and number of cell-cell adhesions are important for maintaining tissue stiffness and mechanical integrity. Although this is a simplified model, it demonstrates how computational modeling can be used to study the effects of weakened or lost cell-cell adhesions on tissue mechanics.

Biological Significance: These findings suggest that tissues can withstand small amounts of adhesion loss, but once a critical number of cell-cell connections is lost, their ability to resist stretching decreases rapidly. This may help explain how diseases that weaken or disrupt cell-cell adhesions can lead to reduced tissue strength and impaired function.

Part 2- Lesson 1- From Experiment to Prediction: The Role of Computational Science in Modern Research

Table of contents–Links to each page

NGSS Alignment to SEPs

Lesson Plan

Intro Handout- **Computational Science in Modern Research**

Example Activity for students-Hip Replacement

Your Turn: Reconstructing the Scientific Process

Student Scenario 1–Precision Chemotherapy

Student Scenario 2: The Future of Crash Testing

Student Scenario 3: Designing Stronger Smartphone Glass

Teacher Keys for Tables

NGSS Science Practice Alignment

NGSS Science & Engineering Practice	How This Lesson Addresses the Practice
SEP 2: Developing and Using Models	Students examine how computational models (e.g., finite element, molecular, and machine learning models) are developed from experimental data and used to make predictions. They recognize that models are continually refined as new evidence becomes available.
SEP 4: Analyzing and Interpreting Data	Students analyze experimental, computational, and clinical data to determine how scientists build, evaluate, and improve models. They distinguish between observations, predictions, and conclusions.
SEP 6: Constructing Explanations and Designing Solutions	Students explain how experimental scientists, computational scientists, engineers, and clinicians contribute to solving complex real-world problems. They describe why experimental validation is essential before new technologies or treatments are accepted.
SEP 7: Engaging in Argument from Evidence	Students use evidence from authentic case studies to justify the roles of different research teams, evaluate computational predictions, and determine whether sufficient evidence supports scientific conclusions or engineering decisions.
SEP 8: Obtaining, Evaluating, and Communicating Information	Students read scientific and technical articles, extract relevant information, organize evidence into tables, and communicate how multiple scientific disciplines collaborate to solve complex problems.
Overall SEP Connection	Students discover that modern scientific advances are the result of an iterative process in which experimental scientists collect data, computational scientists build models, engineers and clinicians test predictions, and researchers continually refine models using new evidence. By comparing multiple case studies, students recognize that this collaborative workflow is common across biology, chemistry, engineering, and medicine.

Lesson Plan

Lesson Component	Description	Comments
Lesson Title	**From Experiment to Prediction: The Role of Computational Science in Modern Research**	This is a jigsaw activity
Grade Level	10–12 (Research, Biology, Biotechnology, Engineering, or Computer Science)	24-28 students per class
Time Required	70–90 minutes (or two 45-minute class periods)	The first period would involve the analysis of 1 scenario in groups and the second would be the mixed group and whole class.
Learning Objectives	Students will: • Explain the role of computational science in modern research. • Distinguish among bench, computational, and validation scientists. • Describe how experimental data are used to build computational models. • Explain why computational predictions require experimental validation. • Compare how computational science is used across multiple disciplines.	
Materials	• Student handout: *Computational Science in Modern Research* • One scenario packet per student • Electronic devices for students	
Engage (10–15 min)	Introduce the lesson using the **Hip Implant Example**. Since many do not understand what this is, use this short video	

	https://www.youtube.com/watch?v=Qh0bp2zPEY4 As a class, identify the roles of the bench scientists, computational scientists, and validation scientists. Discuss why computer models must be experimentally validated before they can be trusted.	
Explore (20–25 min)	Divide students into **six groups** Each group receives one scenario. This can be based on where students are sitting, randomly assigned, or due to interest. **Group 1 and 3:** Precision Chemotherapy (HER2/Herceptin) **Group 2 and 5:** Computational Crash Testing **Group 3 and 6:** Designing Stronger Smartphone Glass Students complete the table by predicting the role of each type of scientist using evidence from the reading.	If the “diamond structure” of learning is applied, students can first read their scenario on their own and fill in the chart the best they can before they meet with their groups. Optionally hand out the linked article for the appropriate scenario. Otherwise they need a link and device.
Explain (20–25 min)	Form 6 new groups with one representative from each scenario. Each student teaches their case study by explaining:• the scientific problem• the role of each research team• how computational science contributed• how the model was validated	If time does not allow for this, this can be skipped and the whole class discussion can come next.
Elaborate (10–15 min)	As a class, compare all three case studies. Complete a summary chart identifying:• Experimental data collected• Computational model developed• Validation method• Real-world application Discuss the common	

	workflow shared by all three examples.	
Evaluate (10 min)	**Exit Ticket** 1. Why can't computational models replace experiments? 2. Describe how one computational model was validated. 3. Identify one similarity shared by all three case studies.	
Assessment	**Formative:** Group discussions, completed tables, peer teaching, exit ticket. **Summative (if appropriate):** CER response explaining why modern discoveries require both computational and experimental science using evidence from onevcase study.	
Differentiation	**Support:** Vocabulary list, sentence starters, partially filled in table **Extension:** Research another example, add in the TYPE of computational science used	

Part 2 Lesson 1

Computational Science in Modern Research

Scientific discoveries are often portrayed as the work of a single scientist making an important breakthrough. In reality, many of today's greatest advances result from teams of researchers with very different areas of expertise working together. Experimental (bench) scientists collect data through laboratory experiments, engineers design and test new technologies, computational scientists build mathematical models and computer simulations, and clinical researchers determine whether discoveries improve people's lives.

Computational science has become an essential part of this process. Instead of relying solely on trial and error, scientists can use computers to simulate how bones respond to stress, predict how molecules interact, identify patterns in enormous datasets, or model the properties of new materials before they are ever created. These predictions allow researchers to focus on the most promising ideas, saving time, reducing costs, and accelerating scientific discovery.

However, computer models alone are never enough. Every prediction must be tested against observations or experiments to determine whether the model accurately represents the real world. As new experimental data are collected, scientists improve their models, leading to even better predictions. This cycle of experimentation → modeling → prediction → validation is at the heart of modern scientific research.

In this activity, you will investigate several real-world scientific breakthroughs. For each example, you will work backward from the final achievement to determine which kinds of scientists were involved, what each group contributed, and how experimental and computational research worked together to solve a complex problem. As you complete each case study, look for the common pattern that connects them all.

Computational Science Area	What it does	Example
1. Physics-Based Modeling	Uses physical laws to simulate the real world.	Finite element analysis, fluid dynamics, weather models
2. Molecular modeling	Simulates atoms and molecules	Drug docking, protein folding, catalyst design
3. Data Science and Machine Learning	Finds patterns in large data sets	Cancer genomics, disease prediction, glass composition prediction
4. Systems Modeling	Models interactions among	Ecosystems, metabolic

	many connected parts	pathways, epidemics
5. Optimization and Design	Finds the best solution among many possibilities	Aircraft wings, battery materials, manufacturing processes.

Hip Implant Example: How Bench Science and Computational Science Work Together

Imagine you are about to receive a hip replacement. Before your surgeon ever makes the first incision, engineers already know how the implant is expected to behave. They can predict where the implant will experience the greatest stress, how the surrounding bone will respond as you walk, and whether certain regions of the bone may be at risk of fracture. These predictions help engineers improve implant designs and give surgeons confidence that the implant is safe and reliable.

How is this possible? No one can watch the inside of a person's hip while they walk. Instead, scientists developed a process that combines laboratory experiments with computational modeling. Modern biomedical engineering depends on both approaches.

This process did not begin with computers. It began with traditional laboratory experiments.

Scientist Group 1: Bench Scientists—Understanding How Bone Behaves

Before engineers could build computer models, they first needed to understand how real bone behaves under mechanical forces.

Beginning in the 1960s, experimental (bench) scientists sought to understand how human bone behaves under mechanical forces. They collected bones from cadaver donors and performed carefully controlled laboratory experiments, measuring how much force bone could withstand before bending or fracturing. Over the next two decades, researchers compared different regions of the femur, investigated how bone density affected strength and stiffness, and developed increasingly accurate methods for measuring the mechanical properties of both cortical and cancellous bone. Their goal was simple: measure reality as accurately as possible. Together, these studies established the fundamental mechanical properties of human bone that later became the foundation for computational models.

One example is:

The Mechanical Characteristics of Cancellous Bone at the Upper Femoral Region

Martens et al., Journal of Biomechanics (1983)

Using compression tests on hundreds of bone specimens taken from human femurs, the researchers measured how bone density influenced stiffness and strength. Their work helped establish the material properties that later became essential for computer models.

Scientist Group 2: Computational Scientists—Predicting Reality

Although laboratory experiments provided accurate information about how bone behaves, they also had important limitations. Every new implant design required another prototype, another cadaver specimen, and another series of expensive mechanical tests. Scientists realized there had to be a better way.

Computational scientists built upon decades of work by bench scientists. They used the experimental measurements of bone strength, stiffness, and density, along with CT scans showing the bone's internal structure, to construct a mathematical model of the femur. Without these laboratory measurements, the computer model would have had no way of knowing how real bone behaves.

The researchers created a detailed three-dimensional digital model of the femur and divided it into thousands of tiny pieces called finite elements. Each finite element was assigned the material properties that bench scientists had measured experimentally. The computer then solved the physics governing every element simultaneously, allowing scientists to predict how the entire femur would respond during activities such as walking, climbing stairs, or standing.

Rather than measuring reality directly, the computational model used decades of experimental knowledge to predict how bone would behave under new conditions.

One example is:

Experimental Validation of a Finite Element Model of the Proximal Femur Using Digital Image Correlation and a Composite Bone Model

Dickinson et al., Proceedings of the Institution of Mechanical Engineers, Part H (2011)

The researchers created a three-dimensional finite element model of the femur from CT scans and predicted how the bone would deform during walking. The model estimated where stresses would be greatest and how much different regions of the bone would strain under load.

Scientist Group 3: Validation Scientists—Can We Trust the Computer?

A computer model is only useful if it accurately represents the real world. Before engineers could rely on it to evaluate new implant designs, they first needed to answer an important question:**Can we trust the computer's predictions?** To find out, a third team of scientists returned to the laboratory. They repeated mechanical loading experiments using both intact bones and bones fitted with hip implants. While the bones were under load, they measured tiny surface deformations using **Digital Image Correlation (DIC)**, a technique that produces a detailed map of how the bone stretches and compresses.

At the same time, computational scientists recreated the exact same experiment using their finite element model. The computer predicted where the bone would deform and how much strain would occur throughout the bone.

The researchers then compared the computer's predictions with the laboratory measurements. The two sets of results closely matched, showing that the model accurately represented the mechanical behavior of both the bone and the implanted joint.

One example is:

Experimental Validation of Finite Element Models of Intact and Implanted Composite Hemipelvises Using Digital Image Correlation

Ghosh et al., Proceedings of the Institution of Mechanical Engineers, Part H (2012)

The researchers found excellent agreement between the measured strains and the computer's predictions. Because the model successfully reproduced the results of real laboratory experiments, engineers gained confidence that it could be used to evaluate new implant designs before building physical prototypes.

So what does all of this mean?

Today, engineers routinely use validated finite element models to compare different implant materials, shapes, and positions on a computer. The most promising designs are then manufactured and tested experimentally before advancing to animal studies, clinical trials, and ultimately patient care.

The Scientific Workflow

Development Stage	Bench Scientists	Computational Scientists	Validation Scientists
Understanding Bone Behavior	Perform mechanical tests on real bone specimens. Measure stiffness, strength, density, and deformation.	—	Verify experimental methods and ensure measurements are reproducible.
Building the Model	Provide CT scans and experimentally measured material properties.	Construct a three-dimensional finite element model. Assign material properties and simulate physiological loading conditions.	Compare preliminary predictions with experimental observations and improve the model if necessary.
Validating the Model	Repeat laboratory experiments and measure strain using Digital Image Correlation.	Predict stresses and strains for identical loading conditions.	Compare predictions with experiments. Demonstrate agreement between model and reality. Conclude that the model can be trusted for future predictions.

Your Turn: Reconstructing the Scientific Process

The hip implant example showed how different groups of scientists worked together to solve a complex biomedical problem. Experimental scientists collected data, computational scientists built predictive models, and validation researchers tested whether the computer's predictions matched reality.

Predict the role each group of scientists played. Ask yourself:

- What information first had to be discovered experimentally?
- How could computational models or data analysis help solve the problem?
- How would scientists determine whether the predictions were correct?

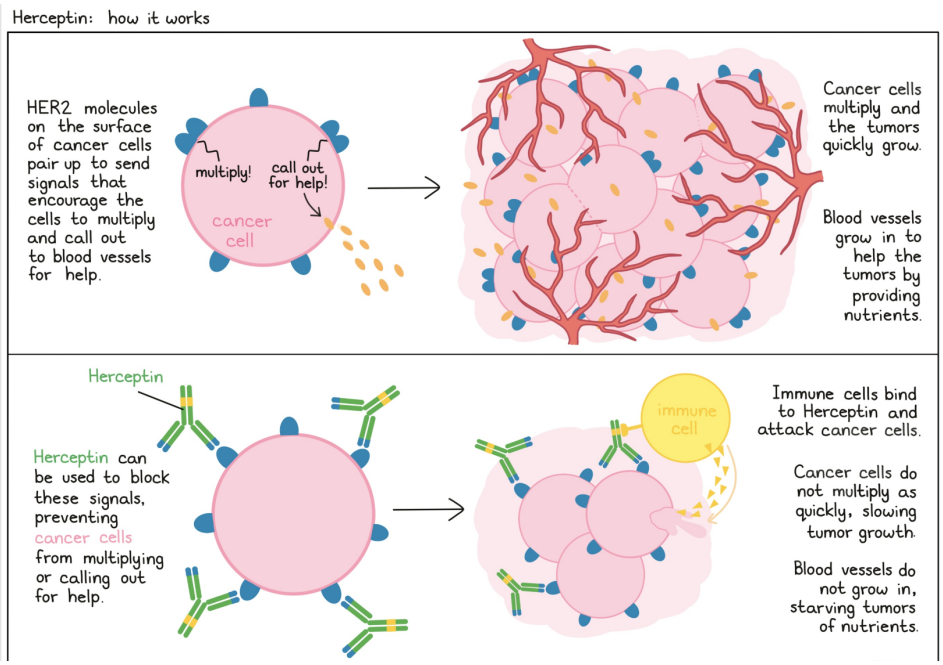
There is no single "perfect" answer. Focus on the logical sequence of research and how different scientific teams worked together. As you complete each scenario, look for the common pattern that connects these discoveries.

Student Scenario 1–Precision Chemotherapy

Cancer is one of the leading causes of death worldwide, and breast cancer is the most commonly diagnosed cancer in women. In the past, doctors often prescribed standard chemotherapy and waited to see whether it worked, exposing many patients to unnecessary side effects from ineffective treatments.

Today, doctors can often predict which therapy is most likely to work by analyzing the genetic and molecular characteristics of a patient's tumor. This approach, called **precision medicine**, has transformed cancer care by improving outcomes while reducing unnecessary treatments.

One of the earliest and most successful examples of precision medicine was Herceptin (trastuzumab), a drug developed to treat HER2-positive breast cancer. About 15–20% of breast cancers produce abnormally high levels of the HER2 protein, causing these tumors to grow and spread more aggressively. The breakthrough took more than 20 years. In 1984, researchers discovered the link between HER2 and aggressive breast cancer. During the late 1980s and early 1990s, scientists developed antibodies that targeted the HER2 protein. Clinical trials throughout the 1990s showed that the drug dramatically improved patient outcomes, leading to FDA approval in 1998. Additional studies expanded its use to early-stage breast cancer in 2006. This life-saving therapy was made possible through the collaboration of bench scientists, computational scientists, and clinical researchers.Here is the story



As you read through the stages in the table, predict what each group of scientists had to do to transform a basic scientific discovery into a life-saving treatment.

Development Stage	**Bench Scientists** ***What experiments or measurements would they perform?***	**Computational Scientists** ***How could computers help solve this problem?***	**Clinical Scientists (Doctors & Clinical Researchers)** ***How would they test whether the treatment helps patients?***
Discovery of HER2 as a problem			
Development of a drug against HER2-positive breast cancer			
Validation of the effectiveness of the drug			

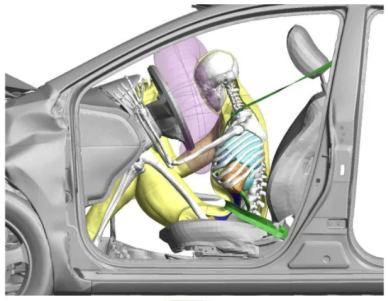
Scenario 2: The Future of Crash Testing

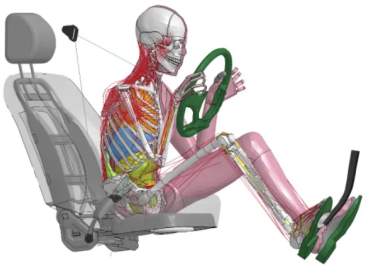
Motor vehicle crashes are one of the leading causes of injury and death worldwide, causing approximately **1.2 million deaths** and **20–50 million injuries** each year. Over the past 50 years, however, advances in vehicle safety have dramatically reduced the risk of death in a crash. Innovations such as seat belts, airbags, crumple zones, and improved vehicle structures have saved hundreds of thousands of lives. Many of these improvements were developed using crash test dummies, which allow engineers to measure the forces experienced by the human body during collisions.

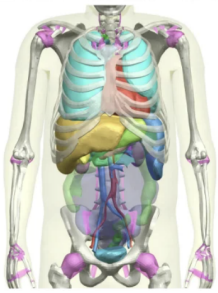
For more than 50 years, automobile manufacturers have relied on instrumented crash test dummies to evaluate vehicle safety. During crash tests, sensors inside the dummies measure forces, accelerations, and body movements, helping engineers design safer seat belts, airbags, and vehicle structures. Traditional crash test dummies, however, have important limitations. They represent only a small number of body types, cannot accurately simulate internal organs or soft tissues, and each full-scale crash test is expensive and destroys an entire vehicle. Engineers also cannot physically test every possible crash speed, collision angle, seating position, occupant age, or body size.

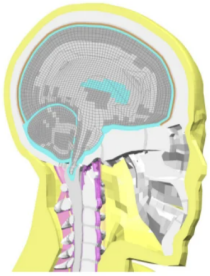
To overcome these limitations, scientists are developing **computational human body models**—highly detailed virtual people built from CT scans, MRI scans, and measurements of bones, muscles, ligaments, and internal organs. These digital humans can simulate thousands of crash scenarios, allowing engineers to predict injuries and evaluate new safety features before a real crash test is ever performed. Before these virtual humans can be trusted, however, scientists must demonstrate that they respond to crashes the same way real people or crash test dummies do. Only after careful experimental validation can computational human body models become reliable tools for designing the next generation of safer vehicles.

. Here is the story.









As you read through the stages below, predict what each group of scientists had to do to transform decades of crash-testing research into realistic virtual humans capable of improving automobile safety

Development Stage	**Bench Scientists / Engineers *What physical data or experiments were needed?***	**Computational Scientists / Engineers*What computer model or simulation was created?***	**Validation Engineers*How would they prove the model is accurate?***
Understanding how the human body responds to crashes			

Development of computational human body models			
Validation of the computer models			

Student Activity 3: Designing Stronger Smartphone Glass

More than **1.5 billion smartphones** are sold worldwide each year, and one of the most common types of damage is a cracked screen. Millions of screens are broken annually, costing consumers **billions of dollars** in repairs and replacements. Developing glass that is both thin and resistant to scratches and fractures has become a major challenge in materials science.

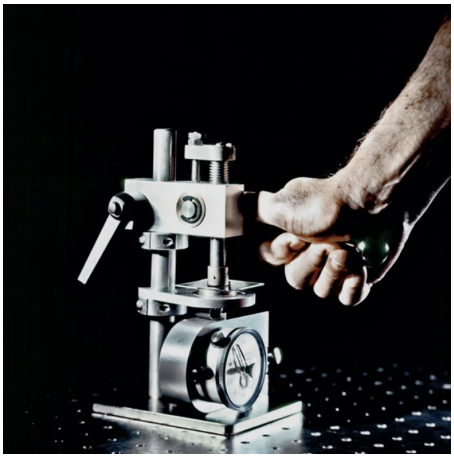
One of the most successful solutions is **Corning® Gorilla® Glass**, a specially engineered glass used in billions of smartphones and other electronic devices.

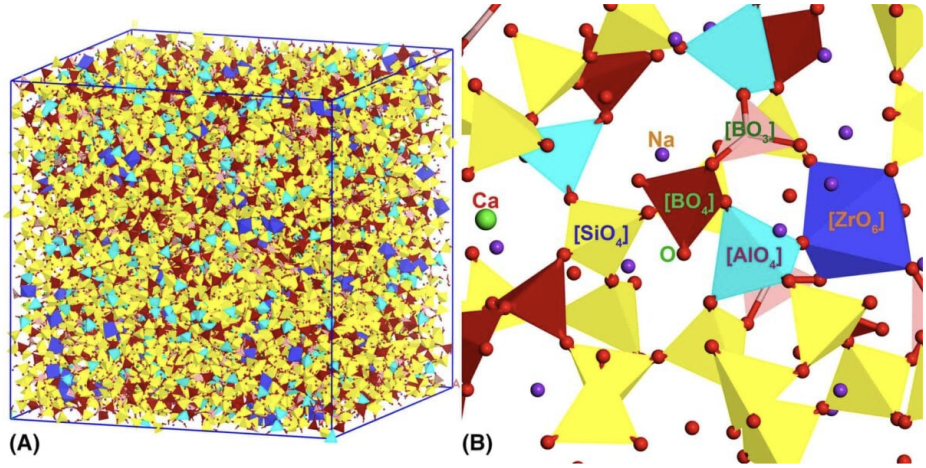
For many years, developing stronger glass was largely a process of trial and error. Chemists created new glass compositions, manufactured samples, and tested each one for strength, scratch resistance, impact resistance, and durability. Because every new composition had to be produced and tested experimentally, developing improved glass often required years of laboratory work.

Today, scientists combine laboratory experiments with **computational materials science** and **machine learning**. Using decades of experimental data, computational scientists build models that predict how changes in chemical composition affect a glass's atomic structure and mechanical properties before it is ever manufactured. This allows chemists to focus on only the most promising glass compositions, saving both time and resources.

Predictions alone, however, are not enough. Scientists must still manufacture the new glass and test it experimentally to determine whether the computer's predictions are correct. Only after careful validation can engineers confidently use the new material in consumer products.

The development of modern **Gorilla® Glass** is the result of collaboration among chemists, computational scientists, materials engineers, and testing engineers, each contributing different expertise to create the stronger, more durable smartphone screens used today. Here is the story.





As you read through the stages below, predict what each group of scientists had to do to transform decades of materials research into the durable glass protecting billions of electronic devices around the world

Development Stage	**Bench Chemists / Materials Scientists*What experiments or measurements would they perform?***	**Computational Scientists*How could computer models help?***	**Validation Scientists / Engineers*How would they prove the new glass works?***
Understanding why smartphone glass breaks			

Designing a stronger glass composition			
Validating the performance of the new glass			

Keys to all Scenarios

Students do not need all answers. One for each box in the table is fine.

Her 2

Development Stage	**Bench Scientists*What experiments or measurements would they perform?***	**Computational Scientists*How could computers help solve this problem?***	**Clinical Scientists (Doctors & Clinical Researchers)*How would they test whether the treatment helps patients?***

Discovery of HER2 as a problem	• Collect breast tumor samples from patients.• Measure HER2 gene amplification and HER2 protein levels.• Compare HER2-positive and HER2-negative tumors.• Study how HER2 affects cancer cell growth.	• Analyze data from hundreds or thousands of patient tumors.• Correlate HER2 status with recurrence and survival.• Determine whether HER2 predicts poor prognosis using statistical and computational analyses.	• Follow patients over many years.• Record treatments, recurrence, and survival.• Provide clinical outcome data for analysis.
Development of a drug against HER2-positive breast cancer	• Develop antibodies that bind specifically to HER2.• Test antibodies on HER2-positive cancer cells.• Evaluate effectiveness in animal models.• Measure whether tumor growth slows.	• Analyze HER2 protein structure and sequence.• Assist in designing and optimizing antibody candidates.• Analyze laboratory results to identify the most promising drug.	• Conduct early clinical trials to determine safe doses.• Monitor side effects.• Evaluate whether tumors respond to treatment.
Validation of the effectiveness of the drug	• Continue laboratory studies to understand how trastuzumab works.• Develop reliable HER2 diagnostic tests (IHC and FISH) so patients can be accurately identified.	• Analyze large clinical trial datasets.• Compare recurrence rates and overall survival.• Identify which patients benefit most from treatment.• Evaluate statistical significance of the results.	• Conduct large randomized clinical trials comparing standard therapy with and without trastuzumab.• Monitor patients for years to measure recurrence, survival, and side effects.• Develop treatment guidelines based on the evidence.

Crash testing

Development Stage	Bench Scientists / Engineers *What physical data or experiments were needed?*	Computational Scientists / Engineers *What computer model or simulation was created?*	Validation Engineers*How would they prove the model is accurate?*
Understanding how the human body responds to crashes	• Collect CT and MRI scans of the human body.• Measure the mechanical properties of bones, muscles, ligaments, and internal organs.• Perform cadaver impact studies and volunteer tests at safe impact levels.• Analyze data from crash test dummies and real-world crashes.	• Organize anatomical and biomechanical data into digital models.• Develop mathematical descriptions of how different tissues deform under impact.• Simulate forces acting on different body regions during a crash.	• Compare computer predictions with measurements from cadaver studies, volunteer tests, and crash test dummies.• Refine the model until it accurately reproduces observed human responses.
Development of computational human body models	• Continue measuring tissue properties under different loading conditions.• Improve anatomical measurements using medical imaging.• Collect injury data from crash investigations.	• Build detailed 3D finite element models of the human body.• Simulate frontal, side, rear, and rollover crashes.• Predict stresses, strains, and likely injuries to bones and organs under different crash conditions.	• Compare predicted body motion and injuries with laboratory crash experiments.• Verify that the model performs accurately for different body sizes, seating positions, and crash types.

Validation of the computer models	• Conduct controlled crash tests using instrumented crash test dummies and vehicles.• Measure forces, accelerations, and vehicle deformation.• Collect additional injury data from real crashes.	• Predict occupant motion, tissue deformation, and injury risk for the same crash conditions tested experimentally.• Update the model as new data become available.	• Compare predicted injuries, body motion, and vehicle deformation with experimental crash results.• Demonstrate that the model consistently matches physical tests before it is used in vehicle design and safety evaluations.

Glass

Development Stage	Bench Chemists / Materials Scientists	Computational Scientists	Validation Scientists / Engineers
Understanding why smartphone glass breaks	Analyze glass composition; measure hardness, fracture toughness, and scratch resistance; study how cracks form after impact.	Analyze existing materials data; model atomic structure and predict how composition affects strength and durability.	Compare model predictions with measurements from existing glass samples.
Designing a stronger glass composition	Prepare new glass formulations; manufacture test samples; measure physical and chemical properties.	Use machine learning and atomistic simulations to predict promising glass compositions before they are made.	Compare predicted properties with laboratory measurements; refine the computational model.
Validating the performance of the new glass	Perform scratch tests, drop tests, and strength measurements; examine fractures using microscopy.	Predict fracture behavior and durability under different conditions.	Compare predicted performance with real drop tests and durability experiments before the glass is used in consumer products.

Part 3: Lesson 2: Before the First Line of Code

Table of contents–Links to each page

[NGSS Alignment to SEPs]

[Lesson Plan]

[Student Handout]

[Student Example]

NGSS Science Practice Alignment

NGSS Science & Engineering Practice	How This Lesson Addresses the Practice
SEP 1: Asking Questions and Defining Problems	Students investigate the driving question, *"What research is required before you can write a computational model?"* They identify gaps in their knowledge, determine what scientific questions must be answered before programming begins, and define the research needed to accurately model a real-world system.

SEP 2: Developing and Using Models	Students analyze an existing computational truss simulation to determine how scientific understanding, mathematical relationships, assumptions, and engineering principles are transformed into a computational model. They recognize that computational models are scientific representations that must be validated and refined before they are useful.
SEP 4: Analyzing and Interpreting Data	Students evaluate what data are required to build and test a computational model, distinguish between observations, model predictions, and assumptions, and discuss how experimental data are used to validate and improve computational simulations.
SEP 5: Using Mathematics and Computational Thinking	Students identify the mathematical relationships, variables, equations, and algorithms needed to simulate a physical system. They discover that computational thinking begins with understanding the science and mathematics underlying the phenomenon before translating those relationships into computer code.
SEP 6: Constructing Explanations and Designing Solutions	Students explain why scientific research, mathematical modeling, engineering concepts, and computational tools must work together to create accurate simulations. They develop a research plan for a computational project by identifying required background knowledge, assumptions, governing equations, and programming tools.
SEP 7: Engaging in Argument from Evidence	During the reverse-engineering activity and gallery walk, students justify the order of research and programming tasks, critique one another's planning organizers, and use evidence to support decisions about what information is necessary before building a computational model.
SEP 8: Obtaining, Evaluating, and Communicating Information	Students identify the scientific literature, engineering principles, mathematical concepts, and data sources they would need to investigate before beginning a computational project. They communicate their research plans and provide constructive feedback to peers during the gallery walk.

Overall SEP Connection	Students discover that computational research is an iterative scientific process in which researchers first investigate the underlying science, identify variables and assumptions, collect or analyze data, develop mathematical models, and only then implement computer code. Through reverse engineering an authentic computational model and planning one of their own, students recognize that successful computational science depends more on understanding the system than on programming itself, reflecting the authentic workflow used by scientists and engineers.

Lesson Plan-Before the First Line of Code

Lesson Component	Description	Comments

Lesson Title	Before the First Line of Code: Research Comes Before Programming	Students discover that computational models are scientific models first and computer programs second.
Grade Level	10–12 (Research Methods, Biotechnology, Engineering, Physics, Computer Science, AP Science, or STEM Research) For me specifically–I want to use this with my research club students who plan to do computational projects. At this time, Research Course students will not have the option of completely computational projects.	Designed for 24–30 students working individually, in small groups, and as a class.
Time Required	50–60 minutes	Can easily be extended into two class periods by having students begin researching their own computational models.
Driving Question	What research is required before you can write a computational model?	This question guides the entire lesson and emphasizes authentic scientific practice.
Learning Objectives	Students will: • Explain why computational modeling begins with scientific research rather than programming.	Aligned primarily with NGSS SEPs 1, 2, 5, 6, 7, and 8.

	• Identify the scientific, mathematical, and engineering knowledge required before creating a computational model. • Differentiate between research tasks and programming tasks during model development. • Describe how assumptions, variables, equations, and data contribute to computational models. • Develop a research plan for a computational modeling project.	
Materials	• Truss/spring simulation • Projector• Reverse-engineering task cards • Sticky notes • Computational Model Planning Organizer • Gallery walk feedback forms (optional)	Optional screenshots showing unsuccessful versions of the simulation illustrate the iterative nature of computational science.

Engage (5–7 minutes)

Lesson Component	Description	Comments

Engage	Without explanation, run the truss/spring simulation for approximately 30 seconds. Ask students, "How many of you think the hardest part of creating this was writing the code?" After students respond, share your personal story of immediately opening Python, believing you could "figure out the science as you went." Explain how this approach repeatedly failed because you lacked sufficient understanding of mechanics, force distribution, assumptions, and mathematical relationships. Show screenshots or examples of failed attempts and describe how each programming challenge revealed a deeper scientific misunderstanding. Conclude by explaining that the biggest lesson learned was that a computational model is a scientific model first and a computer program second. Introduce the driving question: "What research is required before you can write a computational model?" Transition by challenging students to imagine rebuilding the project if every line of code were deleted.	This hook intentionally creates cognitive dissonance by challenging the common misconception that programming is the primary challenge in computational science.

Explore Activity 1 – The Mystery (10 minutes)

Lesson Component	Description	Comments

Explore 1	Students individually respond to the prompt: "What knowledge would someone need before they could create this simulation?" Students record ideas on sticky notes. Common responses include Python, coding, graphics, animation, and mathematics. Collect responses without judging them.	Students reveal their initial perceptions before deeper exploration.

Explore Activity 2 – Reverse Engineer the Project (15 minutes)

Lesson Component	Description	Comments
Explore 2	Divide students into small groups. Each group receives a mixed set of task cards containing research tasks, programming tasks, and distractors (e.g., Learn Hooke's Law, Learn vectors, Understand tension, Find spring constants, Design node data structure, Write a loop, Learn Python syntax, Import NumPy, Debug, Choose colors, Learn recursion, Study bridge engineering, Test model against reality). Students categorize each card (Research, Mathematics, Programming, Validation, etc.) and arrange them in the order they believe they occur during development. Groups justify their reasoning.	There is intentionally no single correct sequence. Discussion focuses on the reasoning behind each group's workflow rather than the "right" answer.

Explain (10 minutes)

Lesson Component	Description	Comments
Explain	Reveal the actual workflow followed while developing the truss simulation. Compare it with student sequences and discuss: Which tasks surprised you? Which required the most time? Which were not programming? Explain that most development time was devoted to learning mechanics, vectors, assumptions, and validation—not writing code. Emphasize that computational modeling is an iterative process requiring continual refinement as scientific understanding improves.	Reinforces the authentic workflow used by scientists and engineers. Alternatively—I can share Science Fair Papers from previous students

Elaborate (15–20 minutes)

Lesson Component	Description	Comments
Elaborate	Students select a computational project of personal interest (e.g., predicting football scores, modeling disease spread, simulating predator-prey interactions, designing a roller coaster, modeling erosion, optimizing traffic lights, stock market prediction). Using a planning organizer, students identify: • What they want to model • Scientific concepts required • Mathematics needed•	Students apply the lesson's framework to their own interests and begin thinking like computational scientists before programming begins. This can be also chosen as a group.

	Data required • Assumptions • Variables or objects the computer must track • Governing equations • Useful programming tools	

Evaluate – Gallery Walk and Exit Ticket (10 minutes)

Lesson Component	Description	Comments
Evaluate	Choose a computational science topic that interests student and develop a possible research question. Analyze how the topic could be investigated using data, programming, modeling, or simulation. Then create a brief display of your idea and participate in a gallery walk to receive feedback, evaluate classmates' projects, and revise your proposed project.	Encourages reflection on how students' perceptions of computational science changed during the lesson.

Assessment

Lesson Component	Description	Comments
Assessment	Formative: Sticky-note responses, group discussions, reverse-engineering card sort, planning organizer, gallery walk feedback, exit ticket. Summative: Students write a CER (Claim-Evidence-Reasoning) explaining why computational models require scientific research before programming, using evidence from the truss simulation and class activities.	Assessment emphasizes understanding of the computational modeling process rather than programming ability. Summative is optional.

Differentiation

Lesson Component	Description	Comments
Differentiation	Support: Provide vocabulary lists (model, variable, assumption, validation, vector, algorithm), partially completed planning organizers, sentence starters, and reduced card sets. Extension: Students research an authentic computational modeling project (e.g., weather forecasting, molecular modeling, autonomous vehicles, finite element analysis, machine learning) and compare its workflow with the class activity. They may also identify the specific computational methods used (finite element analysis, agent-based modeling, machine learning, differential equation modeling, etc.).	Extensions naturally lead into independent computational research projects and reinforce authentic scientific practices.

Before the First Line of Code Student Handout

Plan Your Own Computational Modeling Project

Choose a system, event, or problem that interests you.

Possible topics include:

- Predicting football scores
- Modeling the spread of a disease
- Simulating predator-prey interactions
- Designing a roller coaster
- Modeling erosion
- Optimizing traffic lights
- Predicting changes in a population
- Modeling the motion of an object
- Investigating climate or weather patterns
- Analyzing financial or stock market data

You may also propose another topic with your teacher's approval.

Computational Model Planning Organizer

1. What do you want to model or predict?

2. What is the main question your model would investigate?

3. Why would this be an important or interesting question to answer?

4. What background topics will you need to research?

Topic	Why you need to understand it

5. What data will be needed to create your model? Likely there is more than this table has room for, but just have 5 data pieces.

Data	Why you is this data needed?	Where might you find it? (if you actually find data, include the URL)

6. Name at least 3 variables that your model should be able to vary.

7. Name at least 3 dependent variables that your model should be able to measure.

8. What programming tools might be useful? (Feel free to look these up, unless you already are familiar with them.)

Examples include Python, spreadsheets, NumPy, pandas, Matplotlib, machine-learning tools, simulation software, or statistical software.

9. How would you know whether the model works? (optional)

10. What would be some limitations of your model? (optional)

Example answer–For teacher use

Before the First Line of Code

Student Handout

Plan Your Own Computational Modeling Project

Project Topic: Modeling How a Wildfire Spreads

1. What do you want to model or predict?

I want to create a computational model that predicts how a wildfire spreads through a landscape.

The model would represent the landscape as a grid containing different materials, such as grass, shrubs, trees, roads, water, and previously burned land. It would simulate how the fire changes over time under different environmental conditions.

2. What is the main question your model would investigate?

How do wind speed, wind direction, vegetation type, moisture level, temperature, and terrain affect the speed and direction of wildfire spread?

3. Why would this be an important or interesting question to answer?

Wildfires can destroy ecosystems, homes, businesses, and infrastructure. They can also create dangerous smoke and force people to evacuate.

A computational model could help scientists and emergency planners estimate where a fire might move and which areas may be at greatest risk. It could also help researchers investigate how changes in weather, vegetation, and land management affect fire behavior.

This topic is interesting because wildfires are controlled by several interacting factors. A fire may spread quickly through dry grass but slowly through moist vegetation. Wind can push flames and burning material in a particular direction, while roads, rivers, or areas without vegetation may slow or stop the fire.

The model could also be used to investigate questions such as:

- How effective are firebreaks?
- How does drought affect fire spread?
- Which vegetation types burn most quickly?
- How does wind change the shape of a fire?
- How might controlled burns reduce future wildfire risk?

4. What background topics will you need to research?

Topic	Why you need to understand it
Combustion	I need to understand the chemical reaction that occurs when fuel burns.
Fire triangle	Fire requires fuel, oxygen, and heat. The model must represent conditions that allow fire to continue spreading.
Heat transfer	Conduction, convection, and radiation can move energy from burning material to nearby fuel.
Vegetation as fuel	Different plants contain different amounts of moisture and burn at different rates.
Fuel load	Areas with more leaves, branches, grass, or dead material may produce more intense fires.
Moisture content	Wet vegetation is generally more difficult to ignite than dry vegetation.
Wind speed and direction	Wind can increase oxygen supply, tilt flames, and carry burning material into new areas.
Temperature and humidity	Hot, dry conditions may allow vegetation to ignite and burn more easily.
Topography	Fires often move differently on flat ground, uphill slopes, and downhill slopes.
Firebreaks	Roads, rivers, cleared land, and other barriers may slow or stop fire spread.
Ecosystem disturbance	Fire can harm organisms, but it can also be a natural and important part of some ecosystems.
Controlled burns	Prescribed fires can reduce accumulated fuel and lower the risk of severe wildfires.
Probability	The model may use probability to determine whether one section of the landscape ignites another.

Grid-based models	The landscape could be divided into cells that each have a location, fuel type, moisture level, and fire status.
Model validation	I need to understand how to compare the simulation with experimental or real wildfire data.

5. What data will be needed to create your model?

The complete model may require more data, but four important data categories are listed below.

Data	Why is this data needed?	Where might you find it?
Vegetation or fuel type	Grass, shrubs, and trees may ignite and burn at different rates. The model needs values describing how easily each fuel type catches fire.	Scientific articles, forestry agencies, wildfire research organizations, or controlled classroom experiments using safe substitute materials.
Wind speed and direction	Wind strongly affects the direction and rate of fire spread.	Weather-station records, government weather databases, or classroom fan experiments.
Fuel moisture	Dry vegetation usually ignites more easily than wet vegetation.	Forestry data, drought-monitoring resources, scientific articles, or experiments comparing damp and dry safe materials.
Fire spread rate	The model needs observed measurements to determine whether its predictions are realistic.	Published wildfire reports, fire-science datasets, laboratory studies, or safe computer-based wildfire simulations.

Other useful data could include:

- Air temperature
- Relative humidity
- Slope of the land
- Amount of fuel in each area
- Distance between plants
- Recent precipitation
- Time since the last fire
- Length and location of firebreaks
- Rate at which burning material produces embers

- Duration of burning in each grid cell

6. Name at least three variables that your model should be able to vary.

These are independent variables that the user could change.

1. **Wind speed**
   The model could compare calm conditions with light, moderate, or strong wind.
2. **Wind direction**
   The model could test how the fire changes when wind moves north, south, east, or west.
3. **Vegetation type**
   The model could include grass, shrubs, trees, bare soil, and mixed vegetation.
4. **Fuel moisture level**
   The model could compare dry, moderately moist, and wet vegetation.
5. **Air temperature**
   Higher temperatures may make fuels easier to ignite.
6. **Relative humidity**
   Low humidity may dry vegetation and increase fire risk.
7. **Slope of the land**
   The model could compare flat ground, uphill slopes, and downhill slopes.
8. **Amount of fuel**
   Some areas could contain thick vegetation, while others contain little burnable material.
9. **Starting location of the fire**
   The model could test how the ignition point changes the eventual burned area.
10. **Width and location of firebreaks**
    The model could compare different firebreak designs.

7. Name at least three dependent variables that your model should be able to measure.

These are outcomes that may change when the independent variables change.

1. **Total area burned**
   The model could calculate how many grid cells or square kilometers burn.
2. **Rate of fire spread**
   This could be measured as distance traveled per minute or hour.
3. **Direction of fire spread**
   The model could track the direction in which the fire moves most rapidly.
4. **Time required for the fire to reach a location**
   This could help represent the time available for evacuation.
5. **Maximum width of the fire**
   The model could measure how widely the fire spreads across the landscape.
6. **Percentage of vegetation burned**
   This would show how much of the available fuel was consumed.
7. **Number of structures reached**
   A more advanced model could include fictional buildings or communities.

8. **Number of new fires caused by embers**
   The model could track fires that begin ahead of the main fire front.
9. **Effectiveness of a firebreak**
   The model could measure whether the fire crosses a road, river, or cleared area.
10. **Prediction error**
    The model could compare predicted fire spread with observed results.

8. What programming tools might be useful?

Python could be used as the main programming language.

NumPy could represent the landscape as a two-dimensional grid. Each cell could store information about vegetation, moisture, slope, and whether the area is unburned, burning, or burned.

pandas could organize environmental conditions and the results from repeated simulations.

Matplotlib could create maps or animations showing how the fire changes over time.

Useful graphs might include:

- Total area burned over time
- Wind speed compared with fire-spread rate
- Fuel moisture compared with total area burned
- Firebreak width compared with the probability that the fire crosses it
- Predicted fire spread compared with observed fire spread

SciPy could be used for calculations involving probability, heat transfer, or statistical analysis.

scikit-learn could be used to identify which environmental variables are the strongest predictors of fire size or spread rate.

NetLogo could be useful for creating an agent-based or grid-based simulation without requiring advanced programming.

A **spreadsheet** could be used to organize the first datasets, calculate averages, and compare trials.

A **Jupyter Notebook** could combine explanations, code, maps, calculations, and graphs in one document.

9. How would you test whether the model works? (this would be for students who are planning an actual computational project)

First, I would create a simplified landscape divided into grid cells. Each cell would be assigned a fuel type, moisture level, and fire status.

The possible fire states might be:

- Unburned
- Burning
- Burned
- Unable to burn

The model would begin with one or more burning cells. During each time step, the program would calculate the probability that nearby cells ignite.

The ignition probability would depend on variables such as:

- Vegetation type
- Fuel moisture
- Wind speed
- Wind direction
- Distance from the burning cell
- Slope
- Whether a barrier is present

Because probability is involved, I would run each set of conditions many times. One simulation might spread differently from another even when the starting conditions are the same.

I would calculate the average results from at least 20–50 trials for each condition.

I would first test whether the model produces scientifically reasonable patterns:

- Stronger wind should generally increase fire-spread rate.
- Fire should usually spread more quickly in the direction of the wind.
- Dry fuel should ignite more easily than wet fuel.
- Areas with more fuel should generally burn more intensely or for longer periods.
- Firebreaks should reduce fire spread.
- Wider firebreaks should usually be more effective than narrow firebreaks.
- A fire should not continue spreading through water or areas without fuel.

The model could also be compared with a controlled physical model that does not involve actual fire. For example, students could use a paper grid, dice, and probability rules to simulate fire spread by hand.

The computer model and the paper model could then be run using the same conditions. The results could be compared based on:

- Total area affected
- Direction of spread
- Number of time steps
- Percentage of trials in which the fire crosses a barrier

If an appropriate published dataset were available, the model could also be tested against measurements from an actual wildfire.

The data could be divided into:

- **Training data**, used to select or calculate model parameters
- **Testing data**, used to determine whether the final model makes accurate predictions

The model could be evaluated using:

- Difference between predicted and observed area burned
- Difference between predicted and observed spread rate
- Accuracy in predicting the direction of spread
- Accuracy in predicting whether a fire crosses a barrier
- Consistency across repeated trials

I would also compare the model with a simple baseline model. The baseline might assume that the fire has an equal probability of spreading in every direction. The more advanced model should make better predictions because it includes wind, moisture, fuel, and terrain.

10. What would be some limitations of your model?

One limitation would be that real landscapes are more complex than a simple grid.

A second limitation would be that vegetation is not distributed evenly. Two areas labeled “forest” may contain different tree species, plant densities, and amounts of dead material.

A third limitation would be that wind can change speed and direction during a wildfire.

A fourth limitation would be that the model may treat each grid cell as having one fuel type, even though real areas contain mixtures of plants, soil, rocks, and structures.

A fifth limitation would be that moisture varies among leaves, branches, soil, and living plants.

A sixth limitation would be that fire can create its own local weather conditions. Large fires may produce strong winds and rising columns of hot air.

A seventh limitation would be that embers can travel ahead of the main fire and create new fires. A simple model may not represent this accurately.

An eighth limitation would be that roads and rivers do not always stop fires. Wind may carry embers across them.

A ninth limitation would be that terrain is three-dimensional. A two-dimensional grid may not fully represent valleys, hills, and steep slopes.

A tenth limitation would be that human actions can affect fire behavior. Firefighters may build barriers, apply water, remove fuel, or conduct controlled burns.

An eleventh limitation would be that the ignition probabilities may be based on limited data or assumptions.

A twelfth limitation would be that the model may not include changing temperature, humidity, or precipitation.

A thirteenth limitation would be that the model may work well for one ecosystem but not another. Grasslands, forests, shrublands, and wetlands burn differently.

A fourteenth limitation would be that actual wildfires can be affected by unexpected events, such as equipment failure, lightning, falling trees, or sudden weather changes.

A fifteenth limitation would be that the model may become too complicated if too many variables are included. More variables do not automatically make a model more accurate.

Finally, the model would provide possible outcomes rather than exact predictions. Wildfire behavior is highly variable, so results should be reported as probabilities or ranges rather than certainties.